

Experiment-1-Beginner-1

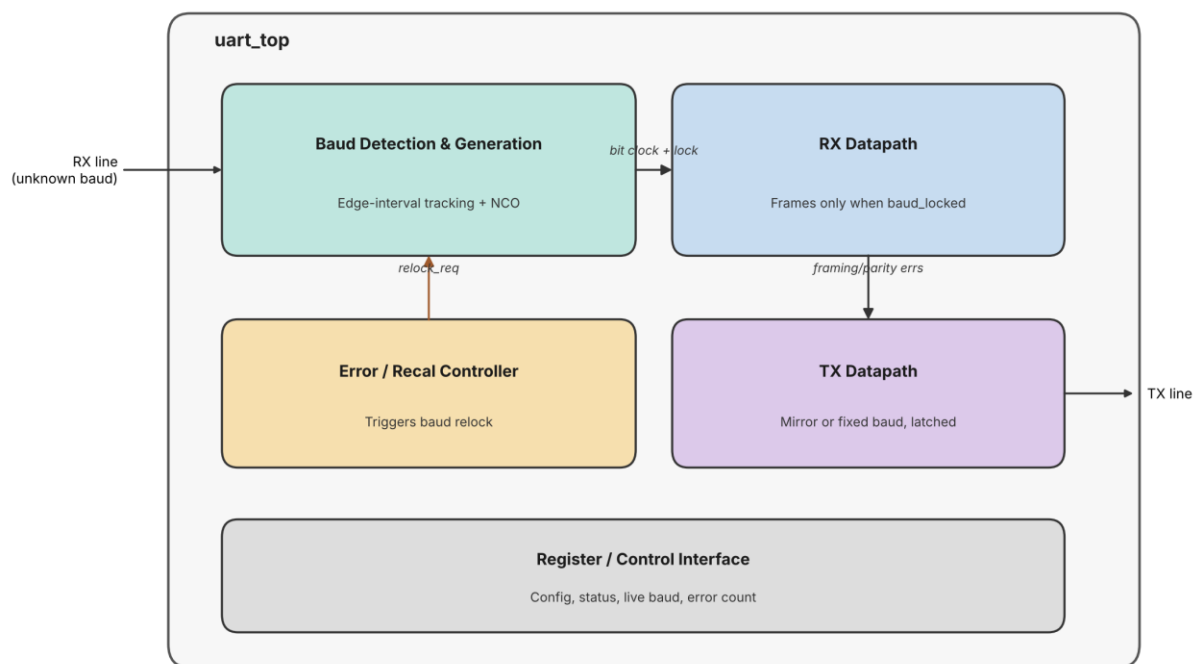
Title: FPGA-Based Self-Correcting UART with Real-Time Baud Detection

Objective:

The objective of this experiment is to design and implement a hardware-oriented, self-correcting UART on an FPGA that automatically detects an unknown incoming baud rate in real time, without the baud rate being configured in advance. The receiver measures the bit period of incoming serial data using edge-interval tracking, generates an oversampled sampling clock through a phase-accumulator NCO, and continuously re-validates its baud estimate as data is received, re-acquiring lock automatically if framing or parity errors indicate the estimate has drifted.

The complete UART core is described using Verilog HDL, validated through ModelSim RTL simulation, and implemented on the Terasic DE10-Nano FPGA board (Cyclone V SE, 5CSEBA6U23I7). The design explores auto-baud acquisition, oversampled receiver synchronization, and self-correcting recalibration to make reliable UART communication possible even when the host baud rate is unknown.

Block Diagram:



The system accepts an asynchronous serial input (RX line) at an unknown baud rate. The line first passes through a synchronizer, after which the Baud Detection & Generation block measures the minimum interval between successive edges to estimate the incoming bit period and uses it to drive an oversampled sampling clock. The RX Datapath samples each bit at its midpoint using this clock, but only frames and delivers a byte once the baud rate has genuinely locked. Framing and parity errors are reported to the Error / Recalibration Controller, which forces the baud detector to re-acquire lock if errors persist. A Register / Control Interface exposes configuration and live status -- detected baud, lock state, and error counts. The TX Datapath transmits using either the same auto-detected baud rate or a fixed configured rate.

RTL Design:

The UART core is implemented entirely in Verilog HDL as a set of cooperating modules, integrated under a single top-level module (uart_top.v). Each block was independently verified before being combined into the complete design.

1. rx_line_sync.v

Provides a double-flop synchronizer for the raw, asynchronous RX pad signal, together with edge-detection pulses used by the rest of the receiver. This is the only block that touches the raw pad signal directly, isolating the rest of the design from metastability risk.

2. baud_rate_detector.v

Implements auto-baud acquisition using a minimum edge-interval tracking method: it continuously measures the shortest interval between edges over a rolling window, filters the result against a configurable tolerance, and declares a lock once a run of consecutive measurements agree. This continuous re-arming is what allows the design to correct for baud drift in real time rather than only at power-on.

3. baud_nco.v

Converts the detected bit period into an oversampling clock using a phase-accumulator (numerically controlled oscillator) architecture, resynchronizing its phase to every genuine start-bit edge so that oversample timing stays consistent from frame to frame.

4. uart_rx.v

Implements the receiver state machine: it samples the synchronized RX line at the midpoint of each bit using the oversampled clock, shifts received bits into a register, and checks framing and parity. The receiver only frames and delivers a byte once the baud rate detector has genuinely locked, so that data received during acquisition is used only to help lock on and is never delivered as if it were valid.

5. uart_tx.v

Implements the transmitter state machine, shifting out start, data, optional parity, and stop bits at the configured or auto-detected bit rate. The selected bit period is latched only between frames, so a mid-transmission update can never distort a byte already being sent.

6. error_controller.v

Aggregates framing, parity, and FIFO overrun errors from the receiver. Once accumulated errors cross a configurable threshold, it issues a relock request back to the baud rate detector, forcing re-acquisition rather than allowing the receiver to continue operating on a baud estimate that may no longer be valid.

7. sync_fifo.v, reg_if.v and uart_top.v

sync_fifo.v is a first-word-fall-through FIFO used for both the receive and transmit data paths. reg_if.v exposes a memory-mapped register interface for configuration and live status. uart_top.v is the top-level integration module that instantiates and wires together every block above into the complete UART core.

8. Testbench and Verification

The integrated design was verified in ModelSim using a top-level testbench, tb_de10nano_uart_top.v, running at a 20 ns (50 MHz) system clock with BIT_CLKS = 434 (approximately 115200 baud). The testbench exercised seven groups of checks: reset behavior, auto-baud acquisition, single-byte echo, multiple-byte echo, and status LED behavior, using self-checking comparisons against expected transmitted and received data.

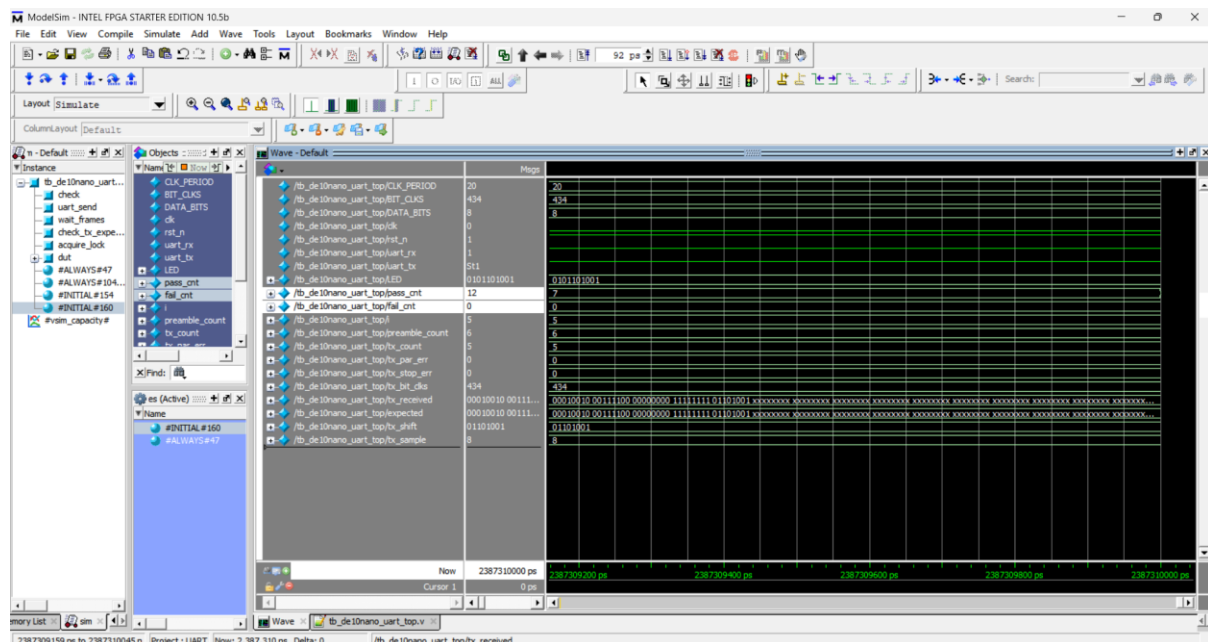
Simulation Results:

```
Transcript
File Edit View Bookmarks Window Help
Transcript
sim:/tb_de10nano_uart_top/1 \
sim:/tb_de10nano_uart_top/preamble_count \
sim:/tb_de10nano_uart_top/tx_count \
sim:/tb_de10nano_uart_top/tx_par_err \
sim:/tb_de10nano_uart_top/tx_stop_err \
sim:/tb_de10nano_uart_top/tx_bit_clks \
sim:/tb_de10nano_uart_top/tx_received \
sim:/tb_de10nano_uart_top/expected \
sim:/tb_de10nano_uart_top/tx_shift \
sim:/tb_de10nano_uart_top/tx_sample
V$IM 3> run -all

# =====
#   tb_de10nano_uart_top: start (BIT_CLKS=434, CLK=20 ns)
# =====

# --- T1: reset state ---
# [2000000] PASS: uart_tx idles high during reset
# [2000000] PASS: LED outputs are clear during reset
#
# --- T2/T3: auto-baud acquisition ---
# [781510000] PASS: LED[8] asserts after auto-baud lock
# [781510000] PASS: Auto-baud acquired within 60 preamble bytes
#
# --- T4/T5: single-byte echo ---
# [1302310000] PASS: LED[7:0] shows last received byte 0xA5
# [1302310000] PASS: Single received byte echoed correctly
# [1302310000] PASS: Echoed UART frame has a high stop bit
#
# --- T6: multiple-byte echo ---
# [2387310000] PASS: LED shows final byte of multi-byte sequence
# [2387310000] PASS: Multiple bytes echoed correctly and in order
# [2387310000] PASS: All echoed frames have high stop bits
#
# --- T7: status LEDs ---
# [2387310000] PASS: Baud-lock LED remains asserted
# [2387310000] PASS: Error LED low after valid frames
#
# =====
# SUMMARY: 12 passed, 0 failed
# *** ALL TESTS PASSED ***
# =====
# * Note: $finish      : C:/Users/pgkri/Downloads/tb_de10nano_uart_top.v(243)
# Time: 2387310 ns Iteration: 0 Instance: /tb_de10nano_uart_top
# 1
```

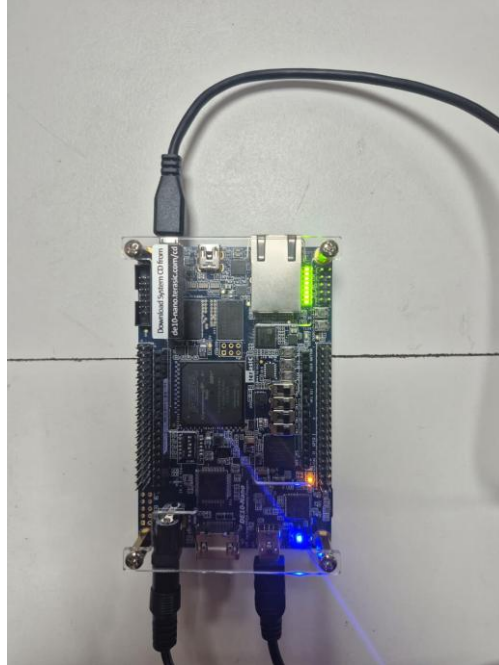
ModelSim transcript: all 12 checks passed across reset, auto-baud acquisition, single- and multi-byte echo, and status LED tests.



ModelSim waveform at end of simulation: tx_received matches expected, LED = 0101101001, pass_cnt = 12, fail_cnt = 0.

The simulation confirmed auto-baud lock (LED[8] asserting) within 60 preamble bytes, correct echo of both single and multiple received bytes with valid stop bits, and a status LED pattern (LED[7:0] = last received byte, LED[8] = baud-lock, LED[9] = error flag) that remained consistent throughout. The final transcript summary reported 12 passed, 0 failed -- all tests passed.

Hardware Implementation:



The design powered up on the Terasic DE10-Nano (Cyclone V SE, 5CSEBA6U23I7).

The verified design was deployed on the Terasic DE10-Nano board. The DE10-Nano provides a 50 MHz system clock (FPGA_CLK1_50 / FPGA_CLK2_50) and two 40-pin GPIO expansion headers (GPIO_0 and GPIO_1) with 36 user-accessible FPGA pins each. The board's onboard USB-UART bridge (FT232R) is wired to the Hard Processor System rather than to the FPGA fabric, so -- as with most pure-fabric UART designs on this board -- the serial RX/TX lines for this design were brought out on the GPIO header to an external 3.3V-logic USB-to-serial adapter for terminal testing, rather than through the onboard HPS USB port. The photograph above shows the board powered and running the design, with the FPGA configured and its status LEDs active.

Applications:

A self-correcting, auto-baud UART is directly applicable wherever the communication baud rate cannot be assumed in advance: embedded debug and bootloader consoles, modular test and measurement equipment that must interoperate with a variety of legacy or third-party devices, IoT sensor nodes with field-configurable host links, and general-purpose FPGA-based communication bridges. The same edge-interval detection and oversampled-NCO architecture generalizes to any asynchronous serial protocol requiring runtime clock recovery, beyond UART specifically.

Conclusion:

The experiment successfully implemented a self-correcting UART core with real-time auto-baud detection, verified in ModelSim with a self-checking testbench covering reset, acquisition, echo, and status behavior, and deployed on the Terasic DE10-Nano FPGA board. All 12 simulation checks passed, confirming correct auto-baud lock, accurate byte echo, and consistent status reporting. The work provided practical experience in asynchronous clock recovery, phase-accumulator NCO design, and FPGA hardware bring-up.

Experiment-2-Beginner-2

Title: FPGA-Based Congestion-Aware Packet Router with Dynamic Arbitration

Objective:

The objective of this experiment is to design and verify a five-port Network-on-Chip (NoC) router in Verilog HDL for use as the basic building block of a two-dimensional mesh. The router receives 40-bit flits on its LOCAL, NORTH, SOUTH, EAST and WEST ports, stores them in per-port, per-virtual-channel input buffers, determines the output port using dimension-ordered X-Y routing, arbitrates for the crossbar using a round-robin switch allocator, and forwards the flits using wormhole switching.

The router also implements credit-based flow control, so that a flit is forwarded only when the downstream buffer has space. The design is validated through a self-checking ModelSim RTL testbench covering ten test scenarios, and a small top-level wrapper (router_top.v) is provided for demonstration on the Terasic DE10-Nano FPGA board using switches, a push button and LEDs.

Block Diagram:

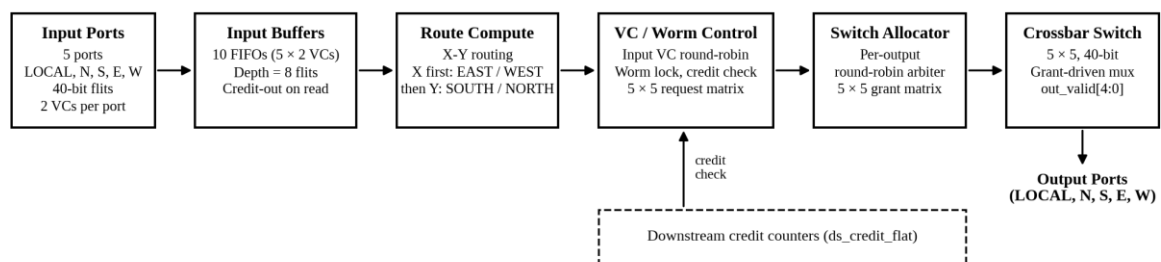


Fig. 1. Overall architecture of the 5-port, 2-VC wormhole NoC router.

A flit entering any of the five input ports is written into the FIFO selected by the one-hot virtual-channel (VC) signal of that port. The head flit at the front of each FIFO is passed to a route-compute unit, which compares the destination coordinates carried in the flit with the router coordinates and produces a one-hot output port. For every input port, the VC and wormhole control logic selects one VC in round-robin order, checks that the requested output VC is not locked by another packet and that the downstream credit counter is non-zero, and raises a request in a 5 x 5 request matrix. The switch allocator grants at most one input to each output port, and the crossbar switch copies the granted flit to the output port and asserts the corresponding out_valid bit. Each flit sent decrements the downstream credit counter of its output port and VC, and each credit returned through ds_credit_flat increments it.

RTL Design:

The router is described entirely in synthesizable Verilog HDL. The RTL consists of the `buffer.v`, `route_compute.v`, `switch_allocator.v`, `switch.v` and `router.v` modules. The testbench `router_tb.v` is used for functional verification, and `router_top.v` is a wrapper used for the board demonstration. The default parameters are a 40-bit flit, 4-bit coordinates, a buffer depth of 8 flits, and 2 virtual channels per port.

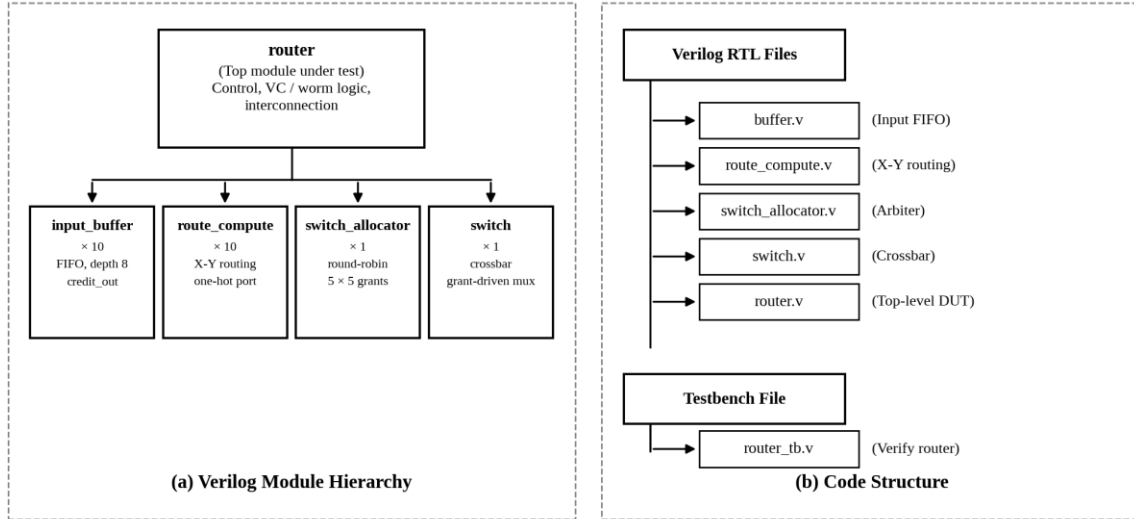


Fig. 2. Verilog module hierarchy and code structure of the NoC router.

The 40-bit flit format used by the router and the testbench is given below. The most significant bit marks the flit as valid, and the head and tail bits delimit a packet so that the router can lock and release a wormhole path.

Bits	Field	Description
[39]	Valid	Always 1 for a flit generated by the testbench
[38]	Head	Marks the first flit of a packet; triggers route computation
[37]	Tail	Marks the last flit of a packet; releases the wormhole path
[36:33]	Destination X	Destination column of the packet
[32:29]	Destination Y	Destination row of the packet
[28:0]	Payload	Data carried by the flit

1. buffer.v

`input_buffer` implements a synchronous FIFO with a parameterised depth (8 flits by default). It uses one extra pointer bit on both the write and read pointers to distinguish the full and empty conditions. A write is accepted only when the buffer is not full, and the read pointer advances only when the buffer is not empty. The flit at the head of the FIFO is always visible on `flit_out`, and `valid_head` is asserted when the buffer is non-empty and the head bit of that flit is set. The `credit_out` signal pulses whenever a flit is read, indicating that one buffer slot has been freed. The router instantiates ten of these buffers (five ports $\times$ two VCs).

2. route_compute.v

`route_compute` extracts the destination X and Y fields of a head flit and compares them with the router coordinates (`ROUTER_X`, `ROUTER_Y`) using signed subtraction. It implements deterministic X-Y routing: if the X offset is positive the output is EAST, if negative WEST; if X matches, a positive Y offset gives SOUTH and a negative Y offset gives NORTH; if both match, the packet is delivered to the LOCAL port. The result is a one-hot 5-bit code (`LOCAL` = 00001, `NORTH` = 00010, `SOUTH` =

00100, EAST = 01000, WEST = 10000). Resolving X before Y makes the routing deterministic and free of routing cycles on a mesh.

3. switch_allocator.v

switch_allocator receives a 5×5 request matrix and produces a 5×5 grant matrix. For every output port it maintains a round-robin pointer and grants the first requesting input starting from that pointer. After a grant, the pointer moves to the input following the winner, so that competing inputs are served fairly and no input is starved.

4. switch.v

switch is the 5×5 crossbar. For each output port, the granted input flit is copied to the corresponding slice of out_flit_flat and the matching bit of out_valid is set. When no grant is present for an output port, its flit is zero and its valid bit is low.

5. router.v

router is the top-level module of the design. It instantiates the ten input buffers, ten route-compute units, the switch allocator and the crossbar, and contains the control logic that connects them. For every input VC it keeps a worm_active flag and the stored output port of the packet currently being forwarded; for every output VC it keeps an out_vc_active lock. A head flit may be forwarded only if its output VC is free; once forwarded, the path is locked until the tail flit has passed, so that flits of different packets are never interleaved on the same output VC. A single-flit packet (head and tail set together) is forwarded without locking the path. Each output port and VC also has a downstream credit counter that is initialised to the buffer depth, decremented when a flit is sent and incremented by ds_credit_flat. A request is raised only if the counter is non-zero, which gives back-pressure when the downstream buffer is full. The VC identifiers are kept unchanged from hop to hop.

6. Testbench and Verification

router_tb is a self-checking testbench. The router under test is placed at coordinate (2,2) with a 10 ns clock and an active-low reset released after 40 ns. Tasks are used to build flits, inject packets on any port and VC, and monitor every output port. The monitor records each flit and VC that appears on the outputs and counts the credits returned on the input side. The expected packet contents, the number of flits at each output port, and the VC are compared automatically, and a [PASS] or [FAIL] message is printed for each check. The ten test scenarios are summarised below.

Test	Stimulus	Checked
TC1–TC4	3-flit packets from LOCAL to routers EAST (3,2), WEST (1,2), SOUTH (2,3) and NORTH (2,1)	Exact flit contents and VC at the expected output only
TC5	2-flit packet from LOCAL addressed to this router (2,2)	LOCAL loopback
TC6	Single-flit packet to EAST followed by a 2-flit packet to WEST	Single-flit worm releases the path
TC7	Two 3-flit packets interleaved on VC0 (to EAST) and VC1 (to WEST)	VC separation and routing
TC8	Two 4-flit packets from LOCAL and NORTH contending for EAST	Both complete without interleaving
TC9	32 back-to-back 4-flit packets from LOCAL to EAST	128 transfers, correct data and order

Test	Stimulus	Checked
TC10	11-flit packet to EAST with downstream credit held at zero, then credits returned	Output stops after 8 flits; buffered flits released by credits

At the end of the run, the testbench also verifies that the number of credits returned by the input buffers equals the number of flits consumed on each input port and VC (163 on LOCAL VC0, 3 on LOCAL VC1 and 4 on NORTH VC0).

Simulation Results:

The RTL was simulated in ModelSim (Intel FPGA Starter Edition 10.5b). The waveform below shows the testbench signals at the instant a flit injected on the LOCAL port (drive_valid = 00001, in_vc_flat = 0000000001, i.e. VC0) leaves the router on the EAST port: out_valid = 01000, with out_vc_flat = 0001000000 identifying VC0 of the EAST output. The destination fields of the flit decode to (3,2), one hop to the east of the router at (2,2), which is consistent with the X-Y routing rule. The downstream credit input ds_credit_flat is all ones in this window, so the output is not stalled.

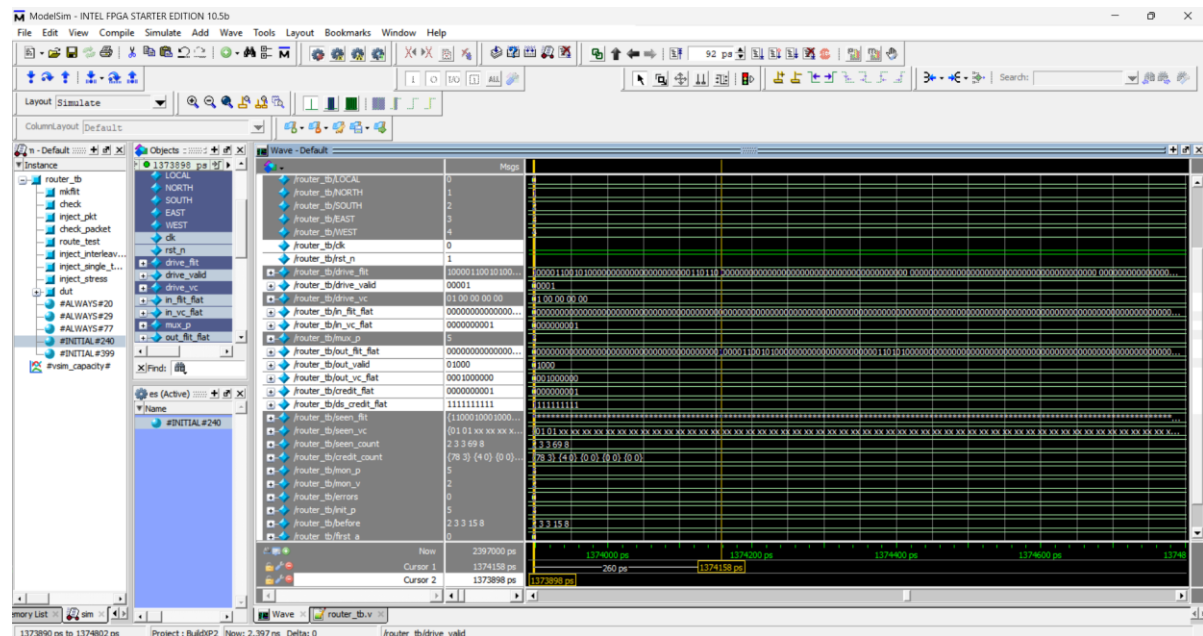


Fig. 3. ModelSim waveform of the router testbench showing a LOCAL-to-EAST transfer.

The transcript below shows the result of the complete run. All checks of TC1 to TC10 and the final credit check are reported as [PASS], the testbench prints ALL TESTS PASSED, and the simulation ends at 2397 ns.

```
Transcript
File Edit View Bookmarks Window Help

sim:/router_tb/out_flit_flat \
sim:/router_tb/out_valid \
sim:/router_tb/out_vc_flat \
sim:/router_tb/credit_flat \
sim:/router_tb/ds_credit_flat \
sim:/router_tb/seen_flit \
sim:/router_tb/seen_vc \
sim:/router_tb/seen_count \
sim:/router_tb/credit_count \
sim:/router_tb/mon_p \
sim:/router_tb/mon_v \
sim:/router_tb/errors \
sim:/router_tb/init_p \
sim:/router_tb/Before \
sim:/router_tb/first_a \
sim:/router_tb/packet \
sim:/router_tb/flit

VSM7> run -all
# ** Error: (vsim-7) Failed to open VCD file ".../sim/router_waves.vcd" in write mode.
# No such file or directory. (errno = ENOENT) : C:/Quartus_Projects/BuildX/router_tb.v(395)
# Time: 0 ps Iteration: 0 Instance: /router_tb
# [PASS] TC1 LOCAL to EAST exact packet
# [PASS] TC2 LOCAL to WEST exact packet
# [PASS] TC3 LOCAL to SOUTH exact packet
# [PASS] TC4 LOCAL to NORTH exact packet
# [PASS] TC5 LOCAL loopback exact packet
# [PASS] TC6 single-flit worm releases EAST
# [PASS] TC6 following packet routes WEST
# [PASS] TC6 no unexpected output
# [PASS] TC7 VCD reaches EAST once per flit
# [PASS] TC7 VC1 reaches WEST once per flit
# [PASS] TC7 no unexpected output
# [PASS] TC8 contending packets both complete
# [PASS] TC8 no unexpected output
# [PASS] TC9 32 back-to-back packets produce 128 transfers
# [PASS] TC9 no unexpected output
# [PASS] TC10 output stops when downstream credits reach zero
# [PASS] TC10 returned credits release buffered flits
# [PASS] TC10 no unexpected output
# [PASS] credit returns match consumed input flits
# ALL TESTS PASSED
# ** Note: $finish : C:/Quartus_Projects/BuildX/router_tb.v(397)
# Time: 2397 ns Iteration: 0 Instance: /router_tb
# Break in Module router_tb at C:/Quartus_Projects/BuildX/router_tb.v line 397
VSM7>
```

Fig. 4. ModelSim transcript showing all test cases passing.

Note: The transcript also contains a message that the VCD file `../sim/router_waves.vcd` could not be opened, because the folder `../sim` does not exist in the project directory. This only affects the optional VCD dump at time 0; it does not affect the checks, and the waveforms were viewed directly in the ModelSim Wave window.

Hardware Implementation:

For demonstration on the Terasic DE10-Nano board, `router_top.v` wraps a single router placed at coordinate (0,0). The board clock and reset drive the router, switches SW[3:2] select the destination X coordinate and SW[1:0] select the destination Y coordinate, and push button KEY1 (active-low, edge-detected) injects a single-flit packet through the LOCAL input on VC0. The downstream credit inputs are tied to zero because there is no external neighbour router, so the internal credit counters (initialised to 8) are the only limit on the number of flits that can leave each output.

The two LEDs display the output direction of the last forwarded packet: 00 for LOCAL, 01 for NORTH, 10 for SOUTH, and 11 for EAST or WEST. Because the destination coordinates from the switches are non-negative and the router is at (0,0), the packet can only be routed to LOCAL, EAST or SOUTH in this demonstration. The FPGA implementation and pin assignment are separate steps handled in Quartus; the results reported in this experiment are from RTL simulation only.



Applications:

The router developed in this experiment is the basic element of a mesh-based Network-on-Chip, which is used to connect processing elements, memory controllers and accelerators in multi-core processors and System-on-Chip designs. Virtual channels and credit-based flow control make it suitable for on-chip communication that requires fair sharing of links, back-pressure without data loss, and reduced head-of-line blocking.

The same router can be instantiated several times to build larger meshes for many-core systems, neural-network and other accelerator arrays, and FPGA-based multiprocessor prototypes, and it can be used as a platform to study routing, arbitration and flow-control techniques.

Conclusion:

The experiment successfully demonstrated a five-port, two-virtual-channel wormhole NoC router with X-Y routing, round-robin switch allocation and credit-based flow control. The RTL design was verified in ModelSim with a self-checking testbench: all ten test scenarios, including contention, back-to-back traffic and credit back-pressure, and the final credit-accounting check passed, and the simulation reported ALL TESTS PASSED. The work provided practical experience in NoC architecture, Verilog RTL design, flow control and simulation-based verification.

Experiment-3-Intermediate-1

Title: FPGA-Based Adaptive-Precision MAC Accelerator for Energy-Efficient Computing

Objective:

The objective of this project is to design and verify a Multiply–Accumulate (MAC) unit in Verilog HDL, which is the fundamental arithmetic operation of digital signal processing and neural-network hardware. The design multiplies two signed operands and adds the product to a running accumulator, using a two-stage pipeline of a multiplier register and an accumulator register.

The design supports selectable operand precision (4-bit, 8-bit, 16-bit and 32-bit) through a precision controller that sign-extends the operands to the 32-bit datapath width. A top-level module provides a board-style interface in which two operands and the precision are entered with four switches and one push button, and the two least significant bits of the result are shown on two LEDs. The design is validated through ModelSim RTL simulation using one testbench for the MAC core and one for the complete top-level module.

Block Diagram:

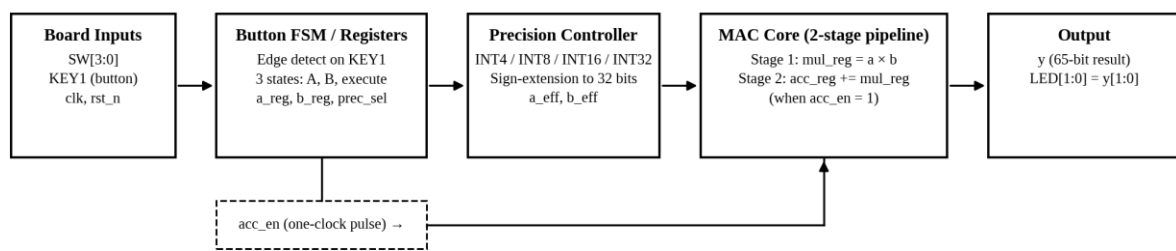


Fig. 1. Overall architecture of the multi-precision MAC unit.

The operands are entered through the four switches SW[3:0] and the push button KEY1. A small three-state controller in the top-level module detects each press of KEY1: the first press captures operand A, the second captures operand B, and the third selects the precision from SW[1:0] and issues a one-clock accumulate-enable pulse (acc_en). The captured operands pass through the precision controller, which selects the lower 4, 8, 16 or 32 bits of each operand and sign-extends it to 32 bits. The MAC core multiplies the two effective operands in its first pipeline stage and, when acc_en is high, adds the registered product to the accumulator in its second stage. The accumulator value is registered at the output y, and its two least significant bits drive LED[1:0].

RTL Design:

The MAC is described in Verilog HDL using the mac_top.v, mac_core.v and precision_ctrl.v modules. Two testbenches are used for functional verification: mac_core_tb.v for the core datapath and mac_top_tb_final.v for the complete top-level design.

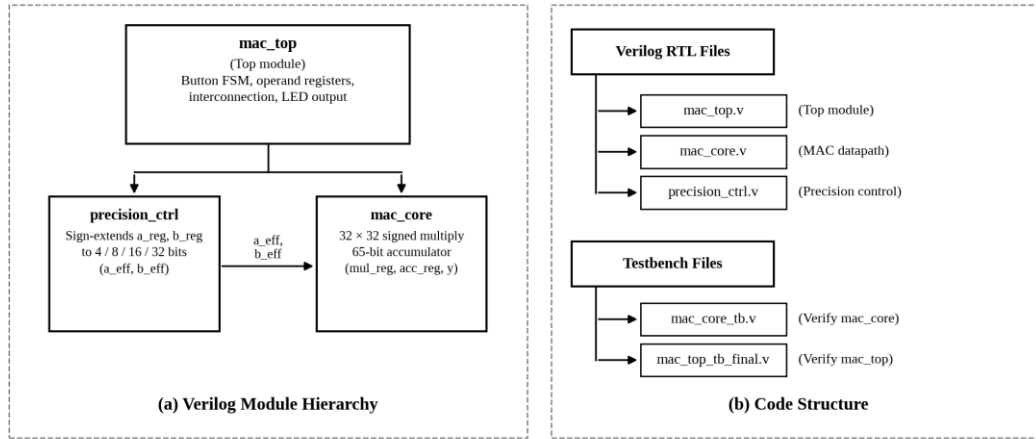


Fig. 2. Verilog module hierarchy and code structure of the MAC unit.

1. mac_core.v

`mac_core` implements the multiply-accumulate datapath with a parameterised data width ($DATA_W = 32$). It takes two signed $DATA_W$ -bit inputs a and b and an accumulate-enable input acc_en , and produces a signed $(2 \cdot DATA_W + 1)$ -bit output y (65 bits by default), which is wide enough to hold the 64-bit product plus one carry bit. The design is a two-stage pipeline. In the first stage the product $a \times b$ is registered in `mul_reg` every clock cycle. In the second stage, when acc_en is high, `mul_reg` is added to the accumulator `acc_reg`. The output y is a registered copy of `acc_reg`, and all registers are cleared by the active-low asynchronous reset. Because of the pipeline registers, the result appears at y a few clock cycles after the operands are applied.

2. precision_ctrl.v

`precision_ctrl` selects the operand width using the 2-bit input `prec_sel`. For 2'b00 (INT4), 2'b01 (INT8) and 2'b10 (INT16), the lower 4, 8 or 16 bits of each input are taken and sign-extended to the full 32-bit width by replicating the most significant bit of the selected field; for 2'b11 (INT32) the inputs pass through unchanged. The outputs `a_eff` and `b_eff` therefore always carry a correctly signed value of the selected precision, so the same 32-bit multiplier is used for all four precisions.

3. mac_top.v

`mac_top` is the board-level top module. It instantiates the precision controller and the MAC core and implements a three-state demonstration controller driven by rising-edge detection of `KEY1`. In state 0 a press stores the four switch values into operand register `a_reg`; in state 1 a press stores them into `b_reg`; and in state 2 a press loads `prec_sel` from `SW[1:0]`, issues a one-clock `acc_en` pulse, and returns to state 0. The `LED[1:0]` output is updated every cycle from the two least significant bits of the MAC result, and the registers are cleared by the active-low reset. Since the operands come from four switches, the values are limited to 0–15 on the board.

4. Testbench and Verification

`mac_core_tb` drives the MAC core directly with a 10 ns clock. After releasing reset it applies the operand pairs 3×4 , 2×5 and 1×6 with `acc_en` toggled, so that the accumulation and the hold behaviour of the accumulator can be observed on y . This testbench has no automatic checks; the output is verified from the waveform.

mac_top_tb_final verifies the complete top-level design through its board-style ports. After reset, it sets SW = 3 and presses KEY1 to capture A = 3, repeats the step to capture B = 3, and then sets SW = 0000 (INT4) and presses KEY1 a third time to execute. After waiting five clock cycles for the pipeline, it prints the precision, the effective operands, mul_reg, acc_reg, mac_result and the LED value, and checks automatically that a_reg = 3, b_reg = 3 and acc_reg = 9. A PASS or FAIL message is printed accordingly.

Simulation Results:

The RTL was simulated in ModelSim (Intel FPGA Starter Edition 10.5b). Compilation of all files completed with 0 errors (Fig. 4). Fig. 3 shows the result of mac_core_tb. After reset is released, the products $3 \times 4 = 12$ and $2 \times 5 = 10$ are accumulated, and the output y reaches 12 and then 22. When 1×6 is applied with acc_en held high, the product 6 is added on successive clock cycles, and y rises to 28 and then 34. This confirms the multiply, accumulate and output-register behaviour of the core, including the pipeline delay of the result with respect to the inputs.

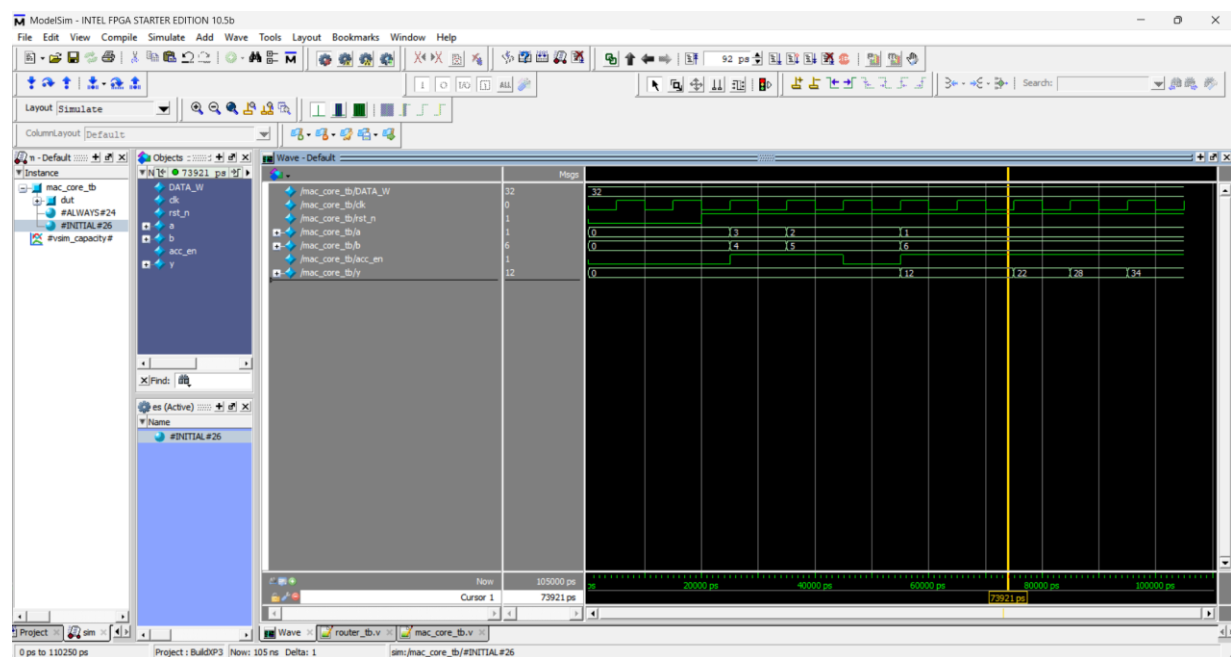


Fig. 3. ModelSim waveform of the MAC core testbench (mac_core_tb).

```

# Compile of mac_core_tb.v was successful.
# Compile of mac_top_tb.v was successful.
# Compile of mac_top.v was successful.
# Compile of mac_core.v was successful.
# Compile of precision_ctrl.v was successful.
# 5 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.mac_core_tb
# vsim -gui work.mac_core_tb
# Start time: 23:41:50 on Oct 10, 2026
# Loading work.mac_core_tb
# Loading work.mac_core
add wave -position insertpoint \
sim:/mac_core_tb/DATA_W \
sim:/mac_core_tb/clk \
sim:/mac_core_tb/rst_n \
sim:/mac_core_tb/a \
sim:/mac_core_tb/b \
sim:/mac_core_tb/acc_en \
sim:/mac_core_tb/y
VSIModel> run -all
** Note: tfinish : C:/Users/pgkri/Downloads/mac_core_tb.v(54)
# Time: 105 ns Iteration: 1 Instance: /mac_core_tb
# 1
# Break in Module mac_core_tb at C:/Users/pgkri/Downloads/mac_core_tb.v line 44
VSIModel>

```

Fig. 4. ModelSim transcript showing successful compilation of the design and testbench files.

Fig. 5 shows the waveform of the top-level testbench. SW is set to 0011 for the first two KEY1 pulses, which capture A = 3 and B = 3, and to 0000 for the third pulse, which selects INT4 precision and executes the MAC operation. After the pipeline delay, LED changes from 00 to 01, which corresponds to the two least significant bits of the result 9 (binary 1001).

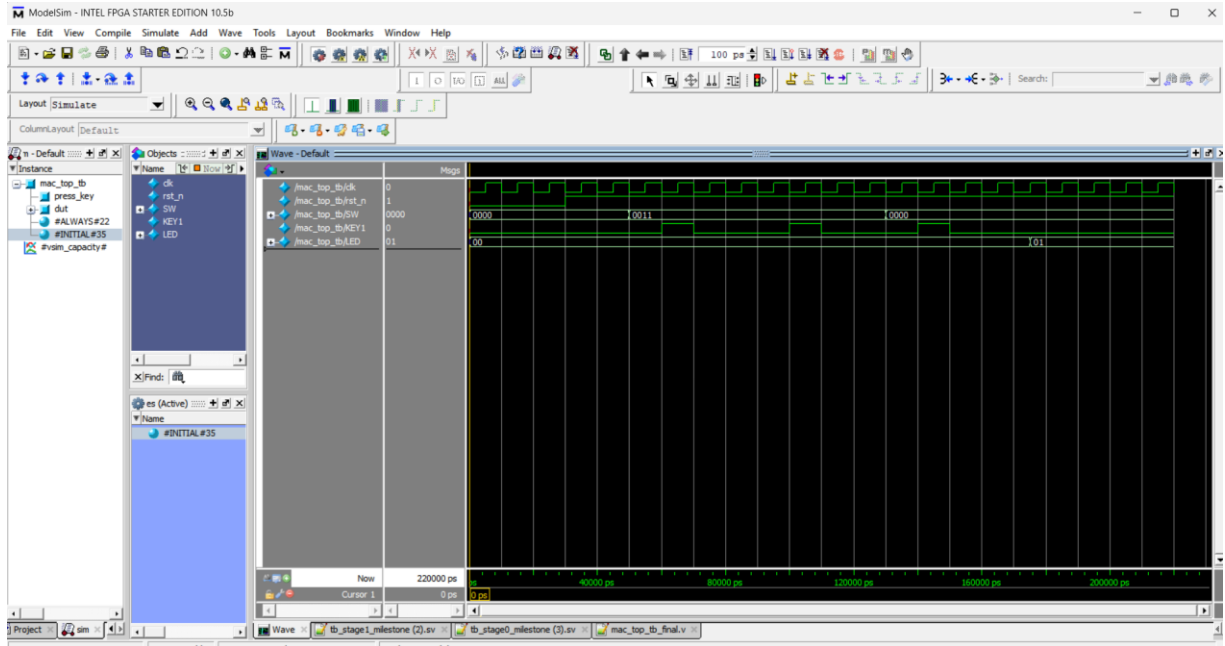


Fig. 5. ModelSim waveform of the top-level testbench (mac_top_tb).

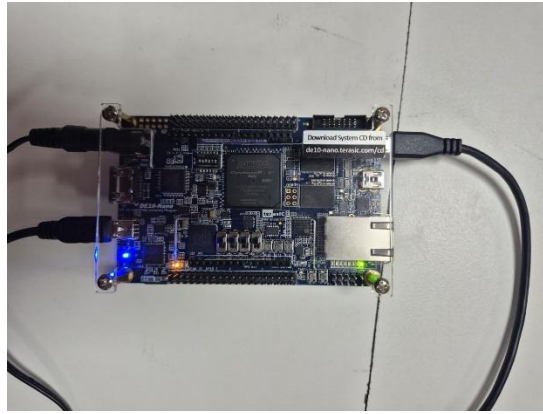
The transcript in Fig. 6 shows the values printed by the testbench: A is captured as 3 at 90 ns and B as 3 at 130 ns. At 220 ns the precision is 00 (INT4) with a_eff = 3 and b_eff = 3, and mul_reg = 9, acc_reg = 9, mac_result = 9 and LED = 01. The testbench prints PASS: 3 x 3 accumulated to 9, and the simulation ends at 220 ns.

```
ModelSim> vsim -gui work.mac_top_tb
# End time: 00:00:37 on Oct 11, 2026, Elapsed time: 0:03:55
# Errors: 0, Warnings: 0
# vsim -gui work.mac_top_tb
# Start time: 00:00:37 on Oct 11, 2026
# Loading work.mac_top_tb
# Loading work.mac_top
# Loading work.precision_ctrl
# Loading work.mac_core
add wave -position insertpoint \
sim:/mac_top_tb/clk \
sim:/mac_top_tb/rst_n \
sim:/mac_top_tb/SW \
sim:/mac_top_tb/KEY1 \
sim:/mac_top_tb/LED
VSI10> run -all
# [90 ns] A captured = 3
# [130 ns] B captured = 3
# [220 ns] precision=00 a_eff=3 b_eff=3
# [220 ns] mul_reg=9 acc_reg=9 mac_result=9 LED=01
# PASS: 3 x 3 accumulated to 9.
# ** Note: $finish : C:/Users/pgkri/Downloads/mac_top_tb_final.v(78)
# Time: 220 ns Iteration: 1 Instance: /mac_top_tb
# 1
# Break in Module mac_top_tb at C:/Users/pgkri/Downloads/mac_top_tb_final.v line 78
```

Fig. 6. ModelSim transcript of the top-level testbench showing the PASS result.

Hardware Implementation:

The top-level module mac_top.v is written with the ports of the target FPGA board (clk, rst_n, SW[3:0], KEY1 and LED[1:0]); the pin assignments are kept in the Quartus project. On the board, operand A and operand B are entered by setting the switches and pressing KEY1 once for each; the third press selects the precision from SW[1:0] and executes the operation. The two LEDs show the two least significant bits of the accumulated result, so for $3 \times 3 = 9$ the LEDs show 01. The results reported in this experiment are from RTL simulation; FPGA compilation and board testing are separate steps.



Applications:

The MAC operation is the core computation of digital filters, convolution, matrix multiplication, and the fully connected and convolutional layers of neural networks. A MAC with selectable precision allows the same hardware to be used at low precision (INT4 or INT8) for energy-efficient neural-network inference and at higher precision (INT16 or INT32) for signal-processing tasks that need a larger dynamic range.

The design can be used as a building block of larger arrays of processing elements, such as systolic arrays and neural-network accelerators on FPGAs, and as a basis for studying pipelining, accumulator sizing and mixed-precision arithmetic in embedded and edge-AI systems.

Conclusion:

The project successfully demonstrated a two-stage pipelined signed MAC unit with selectable INT4, INT8, INT16 and INT32 operand precision and a switch-and-button board interface. The RTL design was verified in ModelSim: the core testbench showed correct accumulation of 12, 22, 28 and 34, and the top-level testbench captured $A = 3$ and $B = 3$, executed in INT4 mode and accumulated 3×3 to 9, with the testbench reporting PASS. The work provided practical experience in Verilog RTL design, pipelined arithmetic, precision control and simulation-based verification.

Experiment-4-Intermediate-2

Title: FPGA-Based Event-Driven Spiking Neural Network (SNN) Accelerator for Handwritten Digit Recognition

Objective:

The objective of this experiment is to design and implement a hardware-oriented Spiking Neural Network (SNN) on an FPGA for handwritten digit classification using the MNIST dataset. The system converts image data into sparse spike sequences using delta encoding, processes the spikes through a fully connected synaptic layer with fixed-point weights, and generates neuron outputs using a Leaky Integrate-and-Fire (LIF) neuron model with shift-based leakage.

The complete SNN is described using Verilog HDL, validated through ModelSim RTL simulation, and targeted for implementation on the Terasic DE10-Nano FPGA board. The design explores the use of sparsity, fixed-point arithmetic, and hardware-friendly neuron computation to make SNN processing suitable for digital FPGA hardware.

Block Diagram:

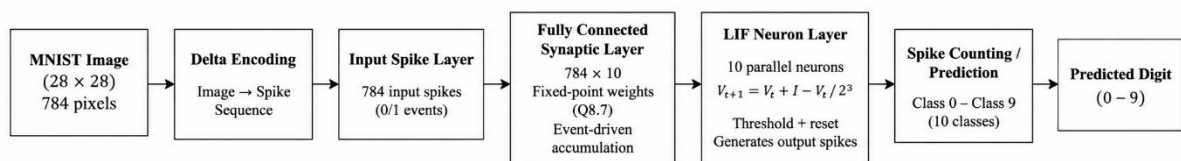


Fig. 1. Overall system architecture of the FPGA-based SNN for MNIST digit classification.

The system accepts a 28×28 MNIST handwritten digit image containing 784 pixels. The image is converted into a temporal sequence of sparse spikes using delta encoding. The resulting spike sequence is supplied to the fully connected synaptic layer, where the active input spikes are multiplied by their corresponding fixed-point weights and accumulated to generate synaptic currents for ten output neurons. Each current is processed by a Leaky Integrate-and-Fire (LIF) neuron. The LIF neurons use shift-based leakage and generate output spikes when their membrane potential reaches the threshold. The output spike activity is used to determine the predicted digit class from 0 to 9.

RTL Design:

The SNN implementation is developed using a combination of Python-based preprocessing and Verilog HDL-based hardware modules. The delta encoding stage is implemented in Python and converts the MNIST images into sparse temporal spike sequences. These generated spike sequences are then sent to the Verilog-based SNN for hardware-oriented processing. The RTL implementation consists of the `snn_top.v`, `fc_layer.v`, and `lif_neuron.v` modules, along with dedicated testbenches `tb_snn_top.v` and `tb_fc_layer.v` for functional verification.

1. `fc_layer.v`

The `fc_layer` module implements the fully connected synaptic layer of the SNN. It accepts a 784-bit input spike vector, where each bit represents the presence or absence of a spike from one MNIST pixel,

and produces ten signed synaptic currents, one for each output digit class. The module stores 7,840 trained weights corresponding to the 784×10 connections. These weights are loaded from the exported memory file and represented using Q8.7 fixed-point format. When an input spike is active, the corresponding weight is accumulated into the respective output current. Thus, the module performs the weighted accumulation required to transform the sparse input spike pattern into ten synaptic currents.

2. lif_neuron.v

The `lif_neuron` module implements the Leaky Integrate-and-Fire neuron model used for spike generation. Each neuron receives a signed synaptic current from the fully connected layer and maintains an internal membrane potential. At every enabled clock cycle, the membrane potential is updated by adding the incoming current and subtracting a leakage term. The leakage is implemented using an arithmetic right-shift operation with `LEAK_SHIFT = 3`, corresponding to division by 2^3 , which provides a hardware-friendly implementation. When the updated membrane potential reaches or exceeds the specified threshold, the neuron generates a one-cycle output spike and resets its membrane potential to zero. Ten instances of this module are used in the complete SNN to represent the ten output classes.

3. snn_top.v

The `snn_top` module is the top-level integration module of the hardware SNN. It connects the `fc_layer` with the LIF neuron layer and controls the flow of the input spike data through the network. The 784-bit input spike vector is provided to the fully connected layer, which generates ten synaptic currents. These currents are then supplied to ten LIF neuron instances, producing ten output spike signals corresponding to digit classes 0–9. The module also provides the clock, reset, enable, and completion signals required for coordinated operation of the SNN.

4. Testbench and Verification

Two testbenches were used for RTL verification. The `tb_fc_layer.v` testbench verifies the functionality of the fully connected layer by providing input spike patterns and observing the generated synaptic currents. The `tb_snn_top.v` testbench verifies the complete SNN by supplying the encoded MNIST spike sequences, processing multiple time steps, and observing the output spikes and predicted digit classes. The complete SNN testbench was used for the final ModelSim evaluation, where 91 out of 100 MNIST test images were classified correctly, giving an RTL simulation accuracy of 91%.

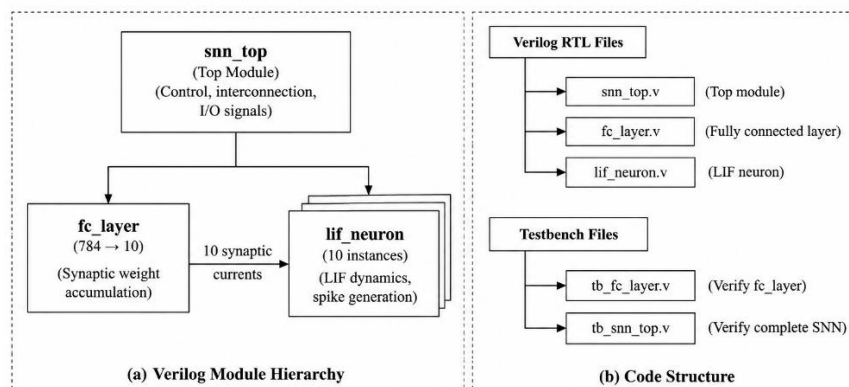
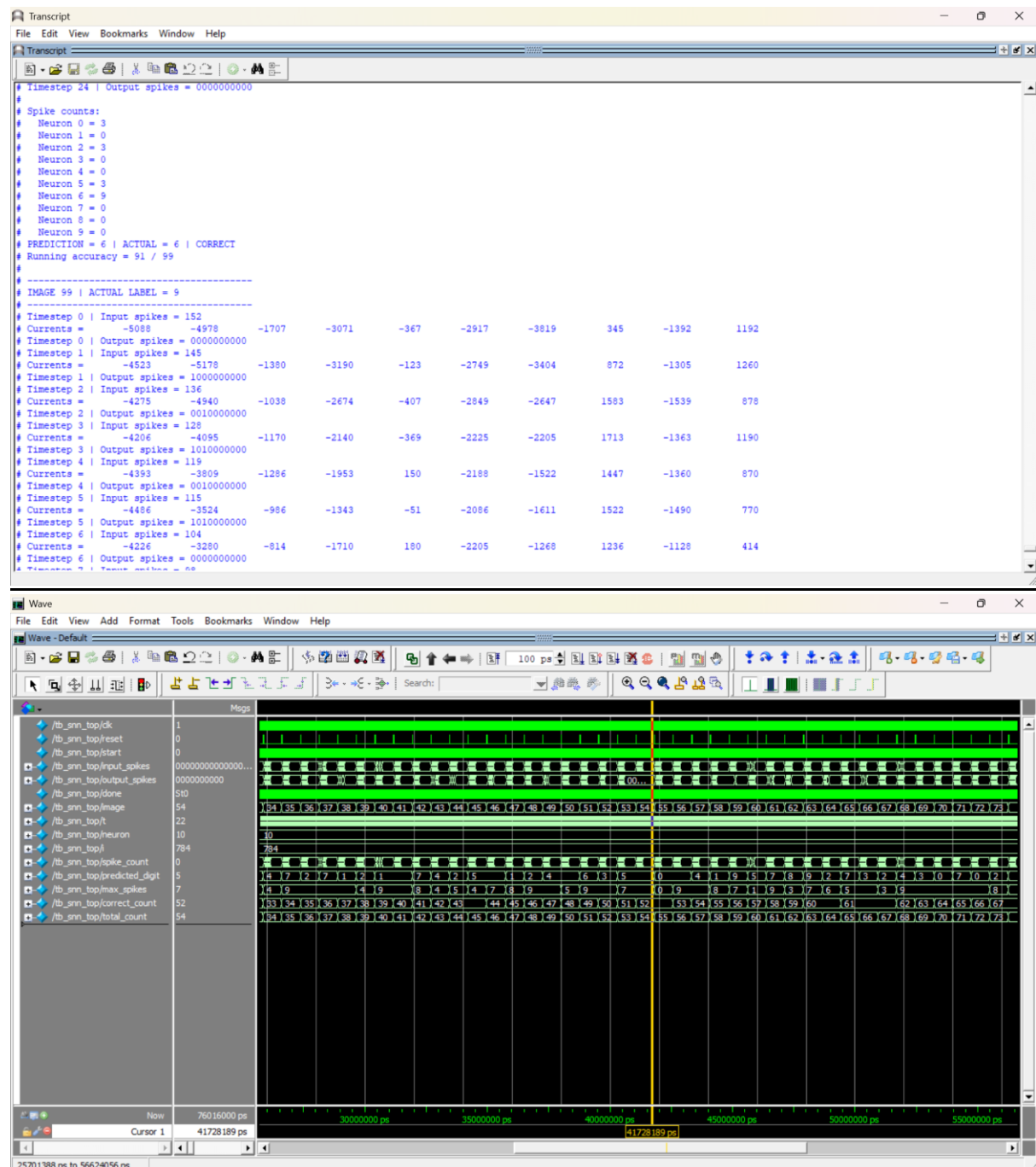


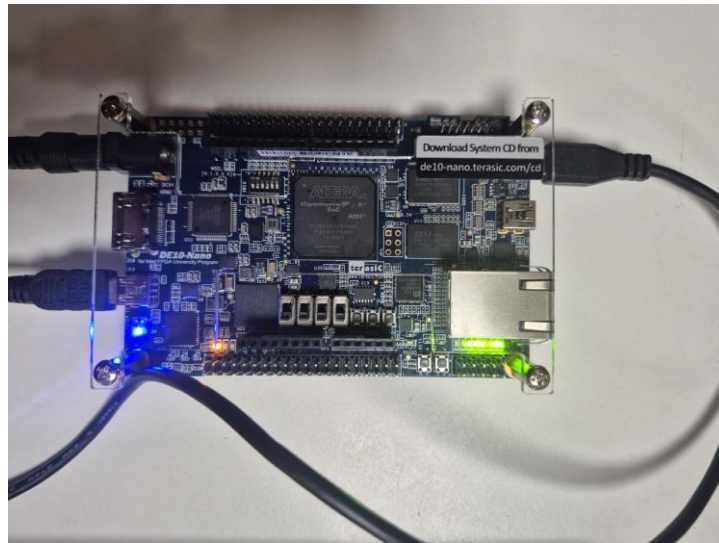
Fig. 2. Verilog module hierarchy and code structure of the FPGA-based SNN.

Simulation Results:



```
=====
100-IMAGE TEST COMPLETE
=====
Correct predictions = 91
Total images       = 100
Accuracy           = 91%
=====
** Note: $finish    : C:/intelFPGA/18.1/tb_snn_top.v(383)
Time: 76016 ns      Iteration: 0   Instance: /tb_snn_top
1
```

Hardware Implementation:



Applications:

The FPGA-based SNN developed in this experiment demonstrates the use of spiking neural computation for image classification on resource-constrained digital hardware. The architecture can be adapted for applications that require event-based processing and low-complexity neural computation, such as handwritten character recognition, edge-based image classification, embedded pattern recognition, and intelligent sensor systems.

The use of sparse spike encoding and hardware-friendly LIF neurons makes the architecture relevant to edge-AI and embedded neuromorphic systems, where neural processing may need to be performed locally rather than relying entirely on a CPU or cloud platform. The same architecture can also serve as a foundation for extending SNN-based processing to other classification tasks and event-driven datasets.

Conclusion:

The experiment successfully demonstrated an FPGA-oriented SNN for MNIST digit classification using sparse spike encoding, fixed-point synaptic processing, and LIF neurons. The RTL design was verified in ModelSim with 91% accuracy on 100 test images. The work provided practical experience in SNNs, Verilog RTL design, fixed-point computation, and FPGA-based neural processing.

Experiment-5-Advanced-1

Title: FPGA-Based Autonomous Hardware Failure Diagnosis and Fault Localization Engine

Objective:

The objective of this experiment is to design and verify an on-chip hardware engine that can detect faults in a digital circuit and localize them to a specific internal net, without an external tester or a human in the loop. The engine uses scan-based testing: the flip-flops of the circuit under test (CUT) are linked into scan chains, a known pattern is shifted in, the circuit is clocked in functional mode, and the captured response is shifted out and compared bit-by-bit with the fault-free (golden) response. Every mismatch is retained in a fail log rather than compressed into a signature, so that the evidence can be traced back to the cause (effect-cause diagnosis).

The design is described in SystemVerilog (IEEE 1800-2012), verified through ModelSim RTL simulation, and targeted for implementation on the Terasic DE10-Nano (Intel Cyclone V SoC) FPGA board. This experiment covers the first two stages of the engine: Stage 0, which checks the integrity of the scan chains themselves, and Stage 1, which narrows the suspected fault site using structural cone intersection. Fault injection hardware is built into the CUT so that stuck-at, transition and scan-chain faults can be created on demand and the diagnosis result can be checked against a known ground truth.

Block Diagram:

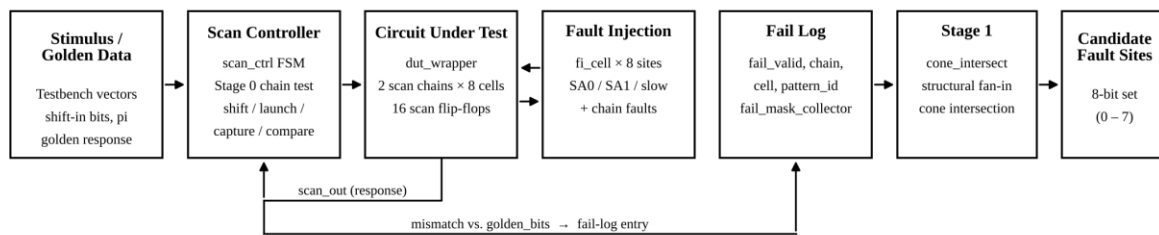


Fig. 1. Overall architecture of the FPGA-based hardware fault diagnosis and fault localization engine (Stage 0 + Stage 1).

The test stimulus (shift-in bits, primary inputs pi) and the expected golden response are supplied to the scan controller. The scan controller first runs a Stage 0 chain test to confirm that every scan chain shifts correctly. It then drives the scan-enable and scan-in signals of the circuit under test, which is a scan-inserted placeholder circuit with 16 flip-flops arranged as two chains of eight cells. Eight fault-injection cells sit on the internal nets of the CUT and can force a stuck-at-0, stuck-at-1 or slow-to-transition fault, while a separate injector can create stuck, broken or slow scan-chain cells. The response shifted out on scan_out is compared with golden_bits every cycle; each mismatch produces a fail-log entry tagged with the pattern, chain and cell. The fail-mask collector gathers these entries into a per-pattern fail mask, and the Stage 1 cone-intersection block intersects the fan-in cones of the failing cells to produce the set of candidate fault sites (0–7).

RTL Design:

The engine is implemented entirely in synthesizable SystemVerilog and is organised as a set of small modules with clear interfaces. Stage 0 is formed by the fi_cell.sv, dut_wrapper.sv, scan_ctrl.sv and stage0_milestone_top.sv modules. Stage 1 adds fail_mask_collector.sv, cone_intersect.sv and stage1_milestone_top.sv on top of the unchanged Stage 0 design. Two self-checking testbenches,

tb_stage0_milestone.sv and tb_stage1_milestone.sv, are used for functional verification. The module hierarchy and code structure are shown in Fig. 2.

1. fi_cell.sv

The fi_cell module is the fault-injection primitive. One instance is placed on each candidate net of the CUT and passes the net value through unchanged unless it is selected. Each instance has a fixed SITE_ID parameter and is selected when fi_active is high and fi_site equals its SITE_ID. The 2-bit fi_type input then chooses the fault: 01 forces the net to 0 (stuck-at-0), 10 forces it to 1 (stuck-at-1), and 11 substitutes a one-clock-delayed copy of the net, which models a slow-to-transition fault. The selection is implemented as a small combinational multiplexer, so it adds no latency when no fault is injected.

2. dut_wrapper.sv

The dut_wrapper module is the circuit under test. It is a hand-designed placeholder with four primary inputs, four primary outputs and 16 scan flip-flops arranged as two scan chains of eight cells (NUM_CHAINS = 2, CHAIN_LEN = 8). Each flip-flop has a 2-to-1 multiplexer in front of it: when scan_en is high, the cell loads the value of the previous cell in its chain (or scan_in for the first cell), and when scan_en is low it loads its functional next-state value. This mux-D structure is the standard scan-insertion scheme. Eight internal nets of the combinational logic are routed through fi_cell instances (NUM_FAULTS = 8). The module also contains a chain-fault injector that can make a selected scan cell stuck, broken or slow, so that faults in the test infrastructure itself can be exercised. The interface is designed so that a scan-inserted ITC'99 b17 benchmark netlist can replace this placeholder later.

3. scan_ctrl.sv

The scan_ctrl module is the test sequencer and the core of Stage 0. It is a finite state machine with the states S_IDLE, S_SHIFT_IN, S_LAUNCH, S_CAPTURE, S_SHIFT_OUT and S_DONE. In chain_test_mode it flushes a repeating 0011 pattern through every chain and compares the output with the input delayed by CHAIN_LEN + 1 cycles; the result is reported per chain on chain_ok, together with the first failing cell on chain_first_bad_cell_flat. In normal mode it shifts the test pattern in with scan_en high, applies one functional capture clock, and shifts the response out while comparing scan_out with golden_bits every cycle. A mismatch raises fail_valid and records fail_pattern_id, fail_chain_id and fail_cell_index. When two_pattern_mode is set, the extra S_LAUNCH state applies a second functional clock, giving the launch-on-capture (V1 → V2) sequence needed to detect transition faults. scan_en is asserted in the same clock edge that leaves S_IDLE so that every shift cycle is a true shift cycle.

4. stage0_milestone_top.sv

The stage0_milestone_top module is the Stage 0 integration level. It contains no logic of its own; it instantiates scan_ctrl and dut_wrapper and connects the scan-enable, scan-in, scan-out and capture signals between them, while bringing the fault-injection controls, test-mode controls and fail-log outputs out to the ports.

5. fail_mask_collector.sv

The fail_mask_collector module converts the serial fail-log stream from scan_ctrl, which reports one mismatching cell per cycle, into a single 16-bit fail_mask per pattern in which each set bit marks a scan cell that captured a wrong value. It also reports whether any cell failed in that pattern. This per-pattern bitmap is the form of evidence that Stage 1 requires.

6. cone_intersect.sv

The cone_intersect module implements Stage 1 fault localization. For every scan cell, a table gives the structural fan-in cone, i.e. the set of fault sites that can logically reach that cell in one functional clock. For each failing pattern, the module takes the intersection of the cones of all failing cells, and across the patterns of a session it intersects these results again, so the 8-bit candidate_sites set can only shrink. A new_session signal resets the set to all ones. Passing cells are deliberately not used to remove candidates, because a fault can be present but masked at a passing cell. Runs made in two_pattern_mode are ignored, since the cone table models a single functional clock. The cone table is hand-derived for the placeholder CUT and will be regenerated for the real benchmark.

7. stage1_milestone_top.sv

The stage1_milestone_top module is the top level of this experiment. It wraps the verified stage0_milestone_top without modifying it and adds the fail_mask_collector and cone_intersect modules, exposing the fail mask and the candidate fault-site set as outputs.

8. Testbench and Verification

Two self-checking testbenches were used for RTL verification. The tb_stage0_milestone.sv testbench first captures the golden response with no fault injected and confirms that a fault-free run produces zero mismatches. It then runs the Stage 0 chain test on healthy chains and on stuck, broken and slow chain cells, and injects stuck-at-0, stuck-at-1 and slow-to-transition faults on the functional nets, checking the number and location of the fail-log entries for each case. The transition fault is run twice: in single-capture mode, where it is expected to be undetectable, and in two-pattern mode, where it must be detected. The tb_stage1_milestone.sv testbench injects a fault at each of the eight sites and checks the candidate set produced by cone intersection, including unique localization, ambiguous sets, a masked fault, multi-pattern sessions, contradictory evidence and the two-pattern guard. Both testbenches were simulated in ModelSim Intel FPGA Starter Edition 10.5b with the -sv option. The Stage 0 testbench reported 8 PASS, 1 EXPECTED and 0 FAIL, and the Stage 1 testbench reported 15 passed and 0 failed.

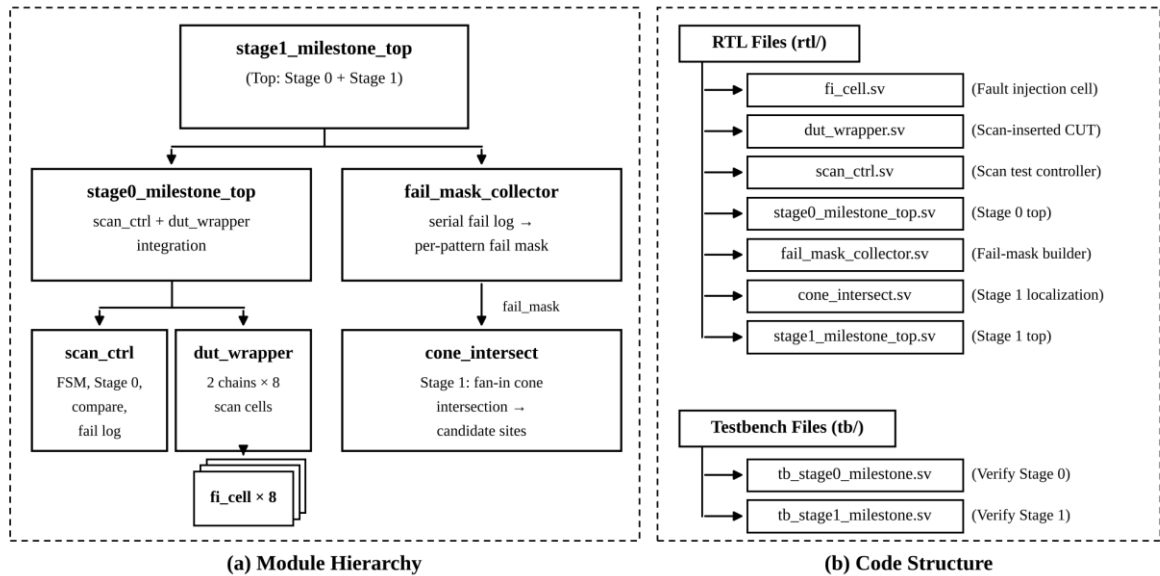
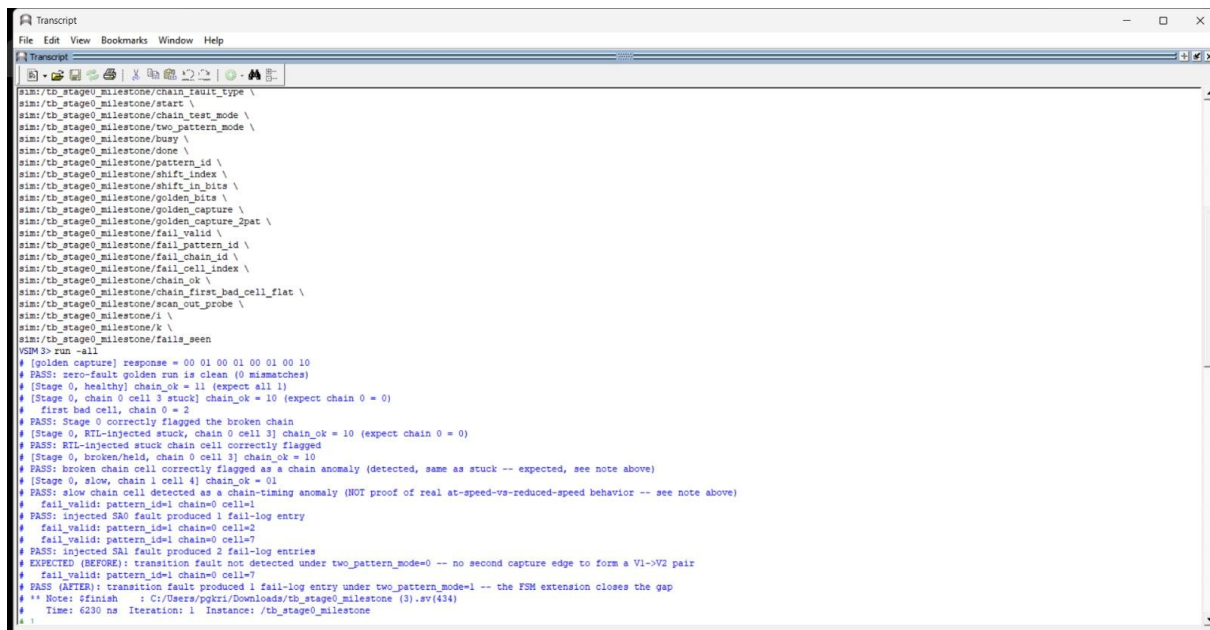


Fig. 2. Module hierarchy and code structure of the fault diagnosis engine.

Simulation Results:



```
Transcript
File Edit View Bookmarks Window Help

sim:/tb_stage0_milestone/chain_fault_type \
sim:/tb_stage0_milestone/start \
sim:/tb_stage0_milestone/chain_test_mode \
sim:/tb_stage0_milestone/two_pattern_mode \
sim:/tb_stage0_milestone/bury \
sim:/tb_stage0_milestone/done \
sim:/tb_stage0_milestone/pattern_id \
sim:/tb_stage0_milestone/shift_index \
sim:/tb_stage0_milestone/shift_in_bits \
sim:/tb_stage0_milestone/golden_bits \
sim:/tb_stage0_milestone/golden_capture \
sim:/tb_stage0_milestone/golden_capture_2pat \
sim:/tb_stage0_milestone/fail_valid \
sim:/tb_stage0_milestone/fail_pattern_id \
sim:/tb_stage0_milestone/fail_chain_id \
sim:/tb_stage0_milestone/fail_cell_index \
sim:/tb_stage0_milestone/chain_ok \
sim:/tb_stage0_milestone/chain_first_bad_cell_flat \
sim:/tb_stage0_milestone/scan_out_probe \
sim:/tb_stage0_milestone/i \
sim:/tb_stage0_milestone/k \
sim:/tb_stage0_milestone/fails_seen

VSM> run -all
[golden capture] response = 00 01 00 01 00 01 00 10
# PASS: zero-fault golden run is clean (0 mismatches)
[Stage 0, healthy] chain_ok = 11 (expect all 1)
[Stage 0, chain 0 cell 3 stuck] chain_ok = 10 (expect chain 0 = 0)
# first bad cell, chain 0 = 2
# PASS: Stage 0 correctly flagged the broken chain
[Stage 0, RTL-injected stuck, chain 0 cell 3] chain_ok = 10 (expect chain 0 = 0)
# PASS: RTL-injected stuck chain cell correctly flagged
[Stage 0, broken/held, chain 0 cell 3] chain_ok = 10
# PASS: broken chain cell correctly flagged as a chain anomaly (detected, same as stuck -- expected, see note above)
[Stage 0, slow, chain 1 cell 4] chain_ok = 01
# PASS: slow chain cell detected as a chain-timing anomaly (NOT proof of real at-speed-vs-reduced-speed behavior -- see note above)
# fail_valid: pattern_id=1 chain=0 cell=1
# PASS: injected SMO fault produced 1 fail-log entry
# fail_valid: pattern_id=1 chain=0 cell=2
# fail_valid: pattern_id=1 chain=0 cell=7
# PASS: injected SAI fault produced 2 fail-log entries
# EXPECTED (BEFORE): transition fault not detected under two_pattern_mode=0 -- no second capture edge to form a V1-V2 pair
# fail_valid: pattern_id=1 chain=0 cell=7
# PASS (AFTER): transition fault produced 1 fail-log entry under two_pattern_mode=1 -- the FSM extension closes the gap
** Note: $finish : C:/Users/pgkri/Downloads/tb_stage0_milestone (3).sv(434)
# Time: 6230 ns Iteration: 1 Instance: /tb_stage0_milestone
```

Fig. 3. ModelSim transcript of tb_stage0_milestone: Stage 0 chain tests and functional fault detection.

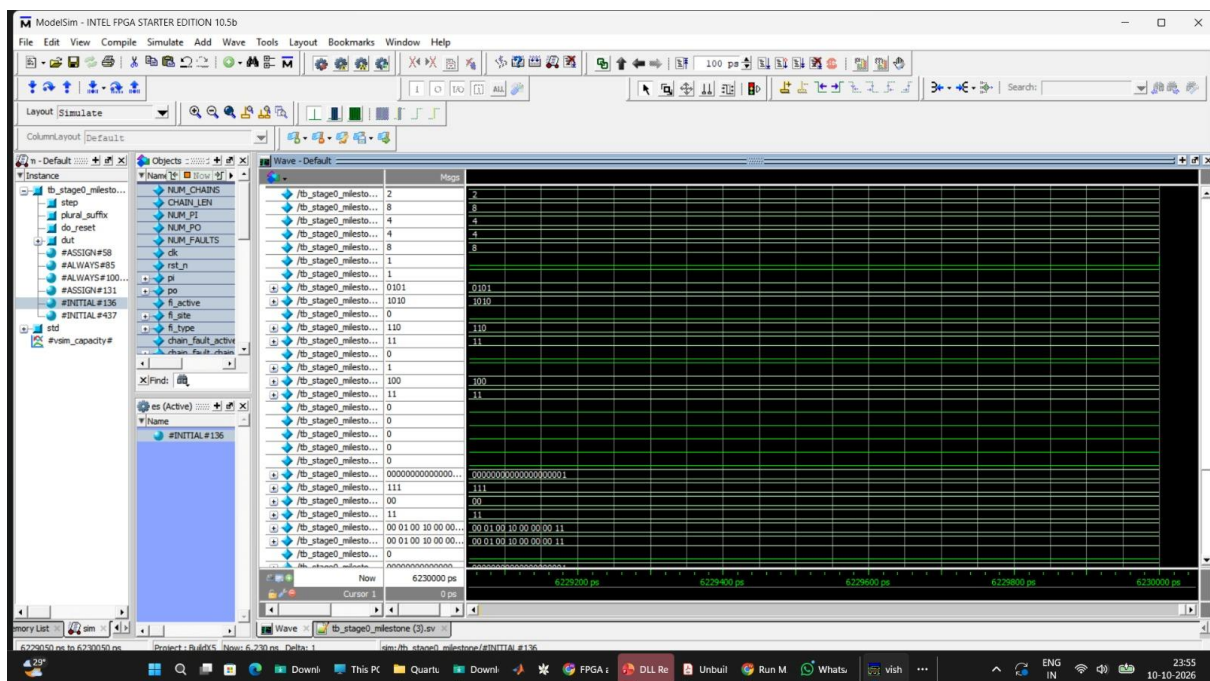


Fig. 4. ModelSim waveform of tb_stage0_milestone.

```

Transcript
File Edit View Bookmarks Window Help
Transcript
sim:/tb_stage1_milestone/po \
sim:/tb_stage1_milestone/fi_active \
sim:/tb_stage1_milestone/fi_site \
sim:/tb_stage1_milestone/fi_type \
sim:/tb_stage1_milestone/start \
sim:/tb_stage1_milestone/chain_test_mode \
sim:/tb_stage1_milestone/two_pattern_mode \
sim:/tb_stage1_milestone/busy \
sim:/tb_stage1_milestone/done \
sim:/tb_stage1_milestone/pattern_id \
sim:/tb_stage1_milestone/shift_index \
sim:/tb_stage1_milestone/shift_in_bits \
sim:/tb_stage1_milestone/golden_bits \
sim:/tb_stage1_milestone/golden_capture \
sim:/tb_stage1_milestone/new_session \
sim:/tb_stage1_milestone/fail_mask \
sim:/tb_stage1_milestone/candidate_sites \
sim:/tb_stage1_milestone/any_cell_failed \
sim:/tb_stage1_milestone/f \
sim:/tb_stage1_milestone/passes \
sim:/tb_stage1_milestone/fails
VSM> run -all
# PASS: clean pattern (no fault) candidates=11111111
# PASS: any_cell_failed=0 on clean pattern
# PASS: site0 SA0 -> unique (0) candidates=00000001
# PASS: site1 SA1 -> unique (1) candidates=00000010
# PASS: site2 SA2 -> unique (2) candidates=00000100
# PASS: site3 SA3 -> unique (3) candidates=00001000
# PASS: site4 SA4 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site5 SA5 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site6 SA6 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site7 SA7 -> ambiguous (2,3,5,7) candidates=10101100
# PASS: site8 SA8 -> masked, nothing learned candidates=11111111
# PASS: session: after site0 SA0 candidates=00000001
# PASS: session: clean pattern doesn't widen candidates=00000001
# PASS: contradictory patterns -> empty set candidates=00000000
# PASS: two-pattern run really failed (fail_mask=00010000000010001), so the guard is what's tested
# PASS: two-pattern_mode ignored by Stage 1 candidates=11111111
#
# Stage 1 summary: 15 passed, 0 failed
# ** Note: cfinish : C:/Users/pgkri/Downloads/tb_stage1_milestone (2).sv(226)
# Time: 9331 ns Iteration: 0 Instance: /tb_stage1_milestone
#
# Break in Module tb_stage1_milestone at C:/Users/pgkri/Downloads/tb_stage1_milestone (2).sv line 226
VSM 7>

```

Fig. 5. ModelSim transcript of tb_stage1_milestone: Stage 1 candidate fault sites (15 passed, 0 failed).

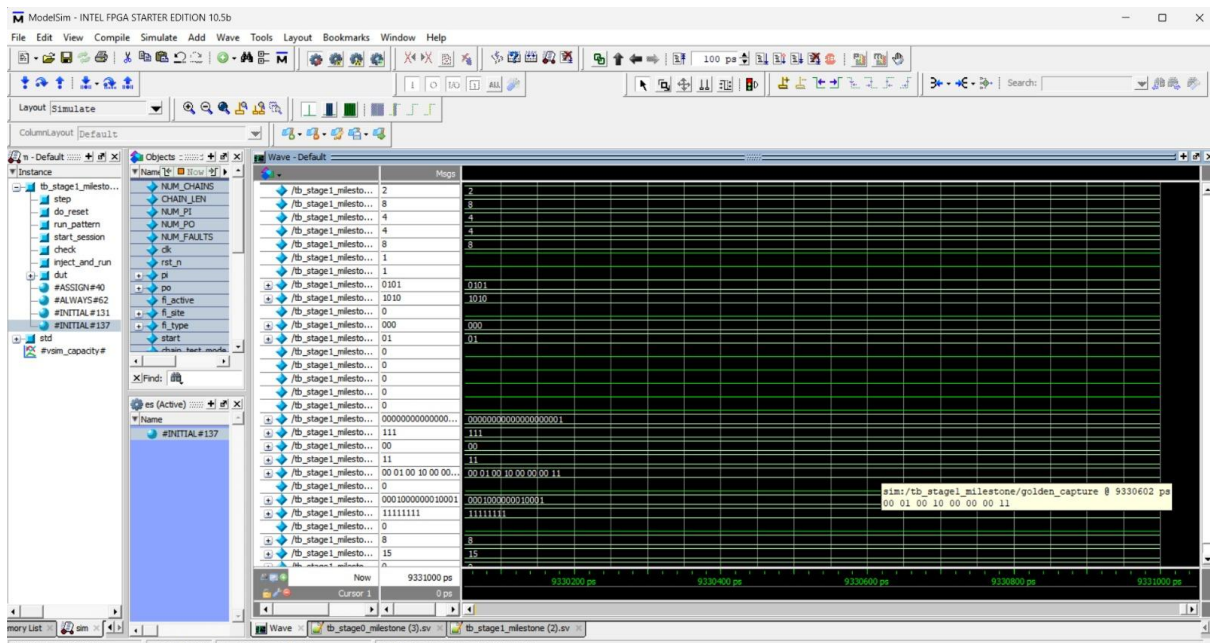


Fig. 6. ModelSim waveform of tb_stage1_milestone.

The Stage 0 results in Fig. 3 show that the fault-free golden run produced no mismatches. A healthy design gives chain_ok = 11, while a stuck cell at chain 0, cell 3 gives chain_ok = 10 with the first bad cell correctly identified; the RTL-injected stuck, broken and slow chain cells are all flagged. A stuck-at-0 fault at site 2 produced one fail-log entry and a stuck-at-1 fault at site 1 produced two. The transition fault was not detected with a single capture clock, as expected, and was detected with one fail-log entry once two_pattern_mode was enabled. The simulation completed at 6230 ns.

The Stage 1 results in Fig. 5 show that a clean pattern leaves all eight sites as candidates (candidates = 11111111). Faults at sites 0, 1, 2 and 3 are narrowed to the single correct site. Faults at sites 4 and 6 are narrowed to {0, 1, 4, 6} and a fault at site 7 to {2, 3, 5, 7}; in each case the true site is inside the set, and separating these sites is the task of the later Stage 2 fault-simulation step. A stuck-at-0 fault at site

5 is masked by the test vector, so no candidates are removed. A clean pattern never widens the set, contradictory patterns empty it, and two-pattern runs are correctly ignored. All 15 checks passed, with the simulation finishing at 9331 ns.

Hardware Implementation:

The design is targeted at the Terasic DE10-Nano development board, which carries an Intel Cyclone V SoC (5CSEBA6U23I7) combining FPGA fabric with a dual-core ARM Cortex-A9 hard processor system (HPS). In the full architecture, Stages 0 to 2 of the diagnosis engine and the circuit under test run in the FPGA fabric at the internal clock rate, while the adaptive Stage 3 diagnosis loop, the software reference model and result reporting run on the ARM processor. Placing the CUT inside the same FPGA removes pin-count and I/O-speed limits on the scan chains and allows faults to be injected at any instrumented net with a known ground truth.

The RTL of this experiment is written in synthesizable SystemVerilog and has passed open-source lint and generic synthesis checks without errors. Compilation in Quartus Prime for the Cyclone V device, pin assignment, timing analysis and on-board testing are the next steps; in the board build, the start, test-mode and fault-injection controls can be mapped to the board's push-buttons and slide switches, and the chain_ok, fail_valid and candidate-site outputs to the user LEDs.

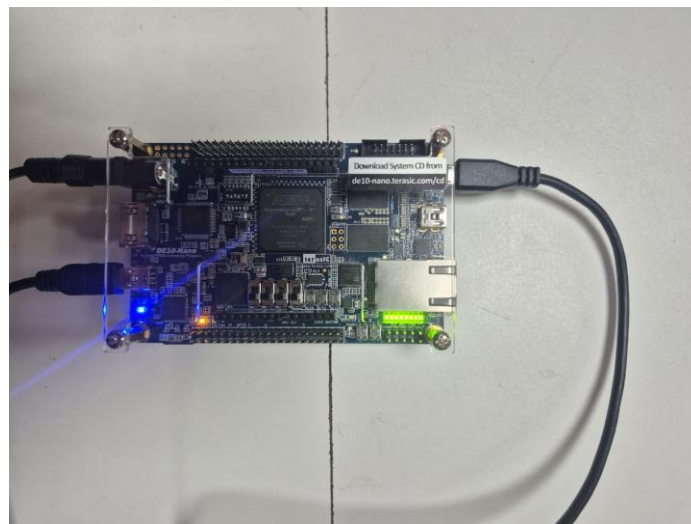


Fig. 7. Terasic DE10-Nano FPGA development board (Intel Cyclone V SoC).

Applications:

The fault diagnosis engine developed in this experiment demonstrates how scan-based testing, fault-log retention and structural fault localization can be built directly into FPGA hardware. The architecture is relevant to production testing and yield learning, where failing parts must be diagnosed down to a specific net so that the physical cause can be found quickly, and to in-field health monitoring of safety-critical electronics, such as automotive, aerospace and medical systems, where a device must detect and report its own faults during idle periods without being removed from service.

The built-in fault injection makes the same platform useful for evaluating test patterns and diagnosis algorithms against known faults, and for teaching design-for-testability concepts such as scan insertion, stuck-at and transition fault models and scan-chain integrity testing. The modular structure also allows the placeholder circuit to be replaced by standard benchmark circuits and extended with fault simulation and adaptive diagnosis stages.

Conclusion:

The experiment successfully demonstrated the first two stages of an FPGA-oriented autonomous fault diagnosis and localization engine. A scan-based test controller with chain integrity testing, two-pattern transition testing and full fail-log retention was designed in SystemVerilog, together with fault-injection hardware and a Stage 1 cone-intersection localizer. The RTL was verified in ModelSim, where the Stage 0 testbench passed all 8 checks with 1 expected outcome and the Stage 1 testbench passed all 15 checks, with stuck-at faults at four of the eight sites localized to the exact net. The work provided practical experience in design-for-testability, scan-chain design, fault modelling, finite-state-machine design and SystemVerilog verification.